Toward a Sparse and Interpretable Audio Codec

John Vinyard

Austin TX, USA
john.vinyard@gmail.com

***Abstract*—Most widely-used modern audio codecs, such as Ogg Vorbis and MP3, as well as more recent "neural" codecs like Meta's Encodec or Descript's are based on block-coding; audio is divided into overlapping, fixed-size "frames" which are then compressed. While they result in excellent reproduction quality and can be used for downstream tasks such as text-to-audio, they do not produce an intuitive, directly-interpretable representation. In this work, we introduce a proof-of-concept audio encoder that represents audio as a sparse set of events and their times-of-occurrence. Rudimentary physics-based assumptions are used to model attack and the physical resonance of both the instrument being played and the room in which a performance occurs, hopefully encouraging a sparse, parsimonious, and easy-to-interpret representation.**

1. INTRODUCTION

Codecs are commonly thought of as transformations into a compressive representation, and while compression is very important, codecs might also transform signals into a representation that is easier to interpret and to manipulate.

We imagine a future audio codec where some types of musical composition could take place directly in the audio codec space. The text-to-audio paradigm works well for non-musicians creating background music for movies, advertisements or social media content, but it is our view that experienced musicians and composers work in a "space" not fully captured by language and that they will prefer a much finer-grained representation that provides multiple scales of interpretability and control.

This work does not seek to produce a generative model of audio, but could serve as the underlying encoding on which generative models are trained. It is the authors' intuition that models trained on this rich, symbolic representation might have a much deeper "understanding" of the content they produce, given the point-cloud-like nature of the signal. Instead of predicting the next frame, the generative model would be predicting the relationships between "events". Long-term coherence in musical generation has always been a difficult problem, and we speculate that many models spend enormous shares of their capacity learning to reproduce physical resonance, rather than the human forces that drive them in interesting, musical directions.

All model and experiment code is implemented using the PyTorch Python library and can be found on GitHub[1]. Audio examples can be heard at this https URL[2]

2. PRIOR WORK

This work draws inspiration from a number of existing techniques, including the field of Auditory Scene Analysis and blind source separation, as well as machine learning techniques like matching pursuit and synthesis techniques like granular synthesis.

- auditory scene analysis - blind source separation - audio source separation - granular synthesis - matching pursuit - source-excitation models (NEEDS A REFERENCE)

[1] https://github.com/JohnVinyard/matching-pursuit
[2] https://blog.cochlea.xyz/sparse-interpretable-audio-codec-paper.html

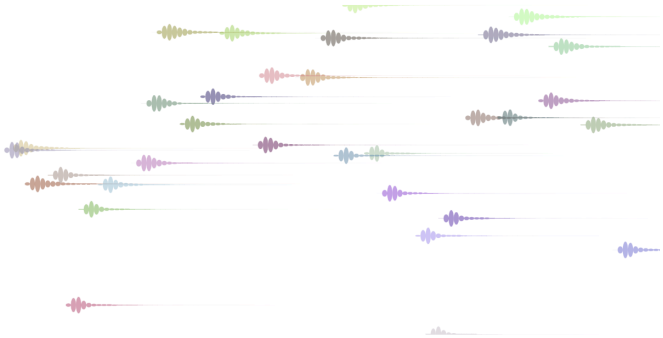

Fig. 1: In this visualization of the codec representation, we see that events can overlap and vary in length. Time is along the x-axis. Event positions along the y-axis and colors are chosen by applying t-SNE to the set of 32 event vectors, targeting a single dimension for the y-axis and three dimensions to represent an RGB color value.

3. AUDIO CODEC DETAILS

Things to mention here are sample-level scheduling and overlapping events.

3.1. Compression Rate

While a high compression rate is not the primary focus of this work, a compressive representation is important and often indicatave of generalization?!?! For our experiment, the encoder is run a fixed number of steps, 32 in our case, and produces a two-tuple of time-of-occurrence and event vector which has 32 dimensions. Each event time can be stored as a single scalar value. Because we encode 2^{16} samples, or around 5.94 seconds of audio at 22050hz, we arrive at:

4. MODEL

4.1. Audio Representation

In this work, we seek an audio representation that can be iteratively decomposed, without encoding perceptually-irrelevant details. For example, a model using "raw" PCM samples as its representation might struggle to remove noisy signals due to undue focus on perceptually irrelevant details.

4.2. Encoder

Inspired by the iterative matching pursuit algorithm [1], the encoding process is iterative, or incremental. At each step, the encoder chooses a single event vector and time-of-occurrence and passes this to the decoder, which generates and schedules audio and subtracts it from the input representation. This process is repeated for some fixed number of steps or until some stopping condition is reached.

In this work, the encoder is paramaterized as an anti-causal, dilated convolutional network, but other types of network, such as UNets or Transformers may yield superior results and will be explored in future explorations.

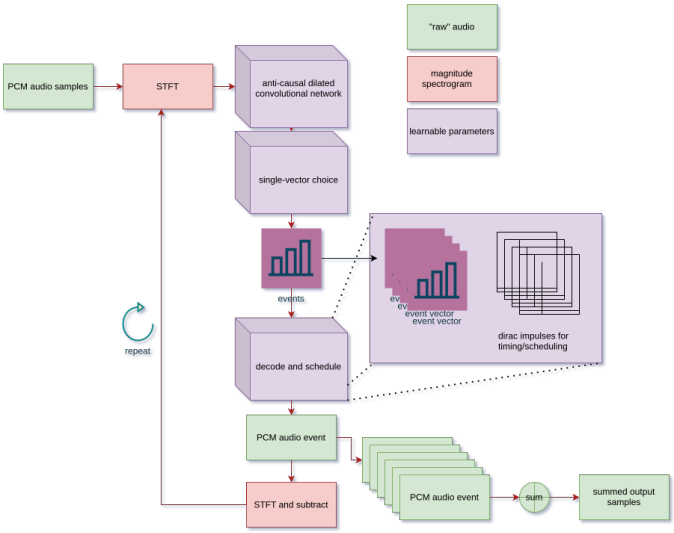

Fig. 2: Model architecture

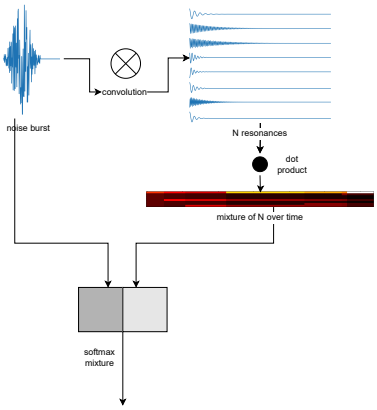

Fig. 3: Decoder Block

4.3. Streaming Algorithm

4.4. Decoder

In order to encourage a sparser and easier-to-interpret representation, we parameterize the decoder as a neural network with the following structure:

Each event is roughly interpreted as a physical process, including the injection of energy into some system and the long convolutions representing the resonance of the sytem and the way the system is deformed over time as it resonates.

TODO: Diagram of decoder block and scheduling process

5. EXPERIMENTS

5.1. Data Set

We train our model on the MusicNet dataset [2], an open dataset containing 33 hours of classical music, recorded in diverse spaces using a range of recording equipment of varying quality. The recordings often include leading silence, trailing applause, and incidental human sounds throughout (coughs, movement, etc).

5.2. Training Algorithm

During training, we repeatedly draw segments of audio that are approximately 5.94 seconds in length from the Music Net dataset at random, with length 2^{17} samples at 22050hz sampling rate and a batch size of two. Each batch is passed through the encoding and decoding process for a fixed number of steps, 32, in our case, and then an iterative loss is computed, encouraging each step to have removed as much energy (expressed as the L1 norm) from the signal as possible. This loss is minimized using the Adam optimizer with a learning rate of $1e^{-4}$.

5.3. Loss Functions

The loss function is also iterative and greedy. Using the same magnitude spectrogram representation observed by the encoder, the loss function attempts to maximize the reduction in the L1 norm of the spectrogram at each step. Prior to computing the loss, the input events are sorted in order of descending norm, such that the event with the most energy is subtracted first and the event with the least energy is subtracted last.

6. INTERPRETABLE AND MANIPULATABLE REPRESENTATION

6.1. Events and Times-of-Occurrence

At the coarse-grained level, overall information density can be gleaned by viewing event times, without regard with event ”contents”.

6.2. Decoder Interpretation

For the decoder used in this set of experiments, intermediate states of the decoder are designed to be meaningful given a source-excitation interpretation of the synthesis process.

7. FUTURE WORK

7.1. Better Perceptual Losses

TODO: Reference to textural loss paper from Josh McDermott

7.2. Decoder Variants

It’s possible that other decoder variants may yield better results, and that events might be best routed to different specialized decoders.

REFERENCES

[1] S. Mallat and Z. Zhang, "Matching pursuits with time-frequency dictionaries," USA, Tech. Rep., 1993.

[2] J. Thickstun, Z. Harchaoui, and S. M. Kakade, "Learning features of music from scratch," in *International Conference on Learning Representations (ICLR)*, 2017.